

Gourmet Bites - Project Documentation & Explanation

This document provides a comprehensive explanation of the **Gourmet Bites** online menu ordering system. You can use this guide to confidently explain how every part of your project works, from the backend logic to the visual frontend design.

1. Project Architecture (Full-Stack)

This project is built using a **Client-Server Architecture**. * **The Server (Backend)**: Written in Python using the Flask framework. It stores the menu data and acts as a calculator for the final bill. * **The Client (Frontend)**: Written in HTML, CSS, and JavaScript. It runs in the user's web browser, displaying the menu, handling user interactions (clicks, typing), and making requests to the server.

2. The Backend: Python & Flask (**App.py**)

The backend is responsible for providing data to the frontend and processing business logic securely.

- **Flask Framework (`app = Flask(__name__)`)**: Flask is a lightweight Python web framework. It listens for HTTP requests coming from the browser and sends back responses.
 - **The Menu Data (**MENU**)**: Instead of a complex database, we used a Python Dictionary to store our items. They are divided into 5 lists: Starters, Beverages, Breads, Main Course, and Desserts.
 - **Endpoints (Routes)**:
 - `@app.route('/')`: The **Home Route**. When the user visits the website, this serves the `index.html` page to their browser.
 - `@app.route('/api/menu')`: An **API Endpoint**. The frontend JavaScript calls this to get the menu data. Flask automatically converts our Python Dictionary into JSON (a format JavaScript can easily read) using the `jsonify()` function.
 - `@app.route('/api/bill', methods=['POST'])`: The **Checkout Endpoint**. When the user checks out, the frontend sends the cart data here. The backend loops through the items, looks up their true prices from the `FLAT_MENU` (to prevent hackers from changing prices in the browser), calculates the subtotal, adds an 8% tax, and returns the final receipt data.
-

3. The Frontend Structure: HTML (**index.html**)

The HTML file acts as the skeleton of the application.

- **Semantic Tags:** We used modern HTML5 tags like `<header>`, `<main>`, `<section>`, and `<footer>` to organize the page logically.
- **Welcome Overlay (`.welcome-overlay`):** This is a full-screen div placed at the top of the body. It forces the user to enter their name before they can see the menu.
- **Dynamic Containers:** Empty `<div>` elements like `<div class="menu-grid" id="menu-grid"></div>` act as placeholders. We use JavaScript to inject the actual food items into these empty boxes later.

4. The Frontend Styling: CSS (`p.css`)

The CSS file gives the project its "premium, cozy café" aesthetic.

- **CSS Variables (`:root`):** We defined our color palette at the very top (e.g., `--primary: #6f4e37;`). If we ever want to change the theme, we only have to change the colors in one place.
- **Glassmorphism:** To make the navigation bar and some containers look like frosted glass, we used the `backdrop-filter: blur(12px);` property along with a semi-transparent background color (`rgba`).
- **Flexbox & CSS Grid:**
 - **Flexbox** was used to align items in a row (like the navigation bar and the cart items).
 - **CSS Grid** was used for the menu (`grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));`). This makes the menu strictly responsive, automatically organizing food cards into rows and columns depending on the screen size.
- **Animations (`@keyframes`):**
 - **heroBgShift:** Slowly transitions the background of the "Savor the Flavor" section through 3 different appetizing food images every 25 seconds.
 - **LogoFade:** Makes the top-left Gourmet Bites logo pulse slightly by changing its opacity.

5. The Frontend Logic: JavaScript (`app.js`)

JavaScript acts as the brain of the browser, making the page interactive without needing to reload.

- **The Welcome Screen:** When the page loads, JavaScript checks for the welcome screen. When the user types their name and clicks "Enter", JavaScript reads the input (`customerNameInput.value`), hides the overlay (`classList.add('hidden')`), and saves the name to use on the final receipt.
 - **Fetching Data (`fetch('/api/menu')`):** As soon as the welcome screen passes, JavaScript makes a background network request to the Python backend to get the food items.
 - **Rendering the Menu (`renderMenu`):** JavaScript loops through the JSON data received from Python. For each item, it creates HTML code on the fly (using `document.createElement`) and injects it into the page. This is why the HTML file is empty at first!
 - **Shopping Cart State (`cart = {}`):** JavaScript uses an object to track what the user has clicked. When you click "Add to Cart", it updates the quantity in this object and immediately updates the visual sidebar.
 - **Event Listeners:** Functions like `.addEventListener('click')` are attached to buttons. They act as "listeners" waiting for you to click so they can trigger actions (like opening the cart or filtering by "Beverages").
-

6. How the "Checkout" Flow Works (The Data Flow)

If your professor or evaluator asks how the whole system connects, explain this flow:

1. **User Action:** The user clicks the "Checkout" button in the shopping cart sidebar.
2. **Frontend Request:** The JavaScript (`app.js`) gathers all items in the cart and packages them into a JSON string. It uses `fetch()` to send a `POST` request to the Python backend (`/api/bill`).
3. **Backend Processing:** Python (`App.py`) receives the cart data. It iterates through the cart, verifies the prices securely, calculates the total with tax, and sends the final receipt data back to the frontend.
4. **Frontend Response:** JavaScript receives the calculated receipt data. It updates the DOM (HTML) of the hidden Receipt Modal (adding the user's name, the tax, and the final total) and changes its CSS class to make it visible on the screen.